Scaling

In the most general sense, **scalability** is defined as the ability to handle more work as the size of the computer or application grows. **scalability** or **scaling** is widely used to indicate the ability of hardware and software to deliver greater computational power when the amount of resources is increased. For HPC clusters, it is important that they are **scalable**, in other words the capacity of the whole system can be proportionally increased by adding more hardware. For software, **scalability** is sometimes referred to as parallelization efficiency — the ratio between the actual speedup and the ideal speedup obtained when using a certain number of processors. For this tutorial, we focus on software scalability and discuss two common types of scaling. The speedup in parallel computing can be straightforwardly defined as

$$Speedup = t(1)/t(N)$$

where **t(1)** is the computational time for running the software using one processor, and **t(N)** is the computational time running the same software with N processors. Ideally, we would like software to have a linear speedup that is equal to the number of processors (speedup = N), as that would mean that every processor would be contributing 100% of its computational power. Unfortunately, this is a very challenging goal for real world applications to attain.

Contents

Scaling tests

As we have already indicated, the primary challenge of parallel computing is deciding how best to break up a problem into individual pieces that can each be computed separately. Large applications are usually not developed and tested using the full problem size and/or number of processor right from the start, as this comes with long waits and a high usage of resources. It is therefore advisable to scale these factors down at first which also enables one to estimate the required resources for the full run more accurately in terms of Resource planning . Scalability testing measures the ability of an application to perform well or better with varying problem sizes and numbers of processors. It does not test the applications general funcionality or correctness.

Strong or Weak Scaling

Applications can generally be divided into strong scaling and weak scaling applications. Please note that the terms strong and weak themselves do not give any information whatsoever on how well an application actually scales. We restate the definitions mentioned in Scaling tests of both strong/weak scaling and elaborate more details for calculating the efficiency and speedup for them below.

Strong Scaling

In case of strong scaling, the number of processors is increased while the problem size remains constant. This also results in a reduced workload per processor. Strong scaling is mostly used for long-running CPU-bound (https://en.wikipedia.org/wiki/CPU-bound) applications to find a setup which results in a reasonable runtime with moderate resource costs. The individual workload must be kept high enough to keep all processors fully occupied. The speedup achieved by increasing the number of processes usually decreases more or less continuously.

In an idealworld a problem would scale in a linear fashion, that is, the program would speed up by a factor of N when it runs on a machine having N nodes. (Of course, as $N \rightarrow \infty$ the proportionality cannot hold because communication time must then dominate. Clearly then, the goal when solving a problem that scales strongly is to decrease the amount of time it takes to solve the problem by using a more powerful computer. These are typically CPU-bound (https://en.wikipedia.org/wiki/CPU-bound) problems and are the hardest ones to yield something close to a linear speedup.

Amdahl's law and strong scaling In 1967, Amdahl pointed out that the speedup is limited by the fraction of the serial part of the software that is not amenable to parallelization. Amdahl's law can be formulated as follows

$$Speedup = 1/(s + p/N)$$

where **s** is the proportion of execution time spent on the serial part, **p** is the proportion of execution time spent on the part that can be parallelized, and **N** is the number of processors. Amdahl's law states that, for a fixed problem, the upper limit of speedup is determined by the serial fraction of the code. This is called strong scaling. In this case the problem size stays fixed but the number of processing elements are increased. This is used as justification for programs that take a long time to run (something that is cpu-bound). The goal in this case is to find a "sweet spot" that allows the computation to complete in a reasonable amount of time, yet does not waste too many cycles due to parallel overhead. In strong scaling, a program is considered to scale linearly if the speedup (in terms of work units completed per unit time) is equal to the number of processing elements used (N). In general, it is harder to achieve good strong-scaling at larger process counts since the communication overhead for many/most algorithms increases in proportion to the number of processes used.

Calculating Strong Scaling Speedup

If the amount of time needed to complete a serial task **t(1)**, and the amount of time to complete the same unit of work with N processing elements (parallel task) is **t(N)**, than Speedup is given as:

$$Speedup = t(1)/t(N)$$

Weak Scaling

In case of weak scaling, both the number of processors and the problem size are increased. This also results in a constant workload per processor. Weak scaling is mostly used for large memory-bound applications where the required memory cannot be satisfied by a single node. They usually scale well to higher core counts as memory access strategies often focus on the nearest neighboring nodes while ignoring those further away and therefore scale well themselves. The upscaling is usually restricted only by the available resources or the maximum problem size. For an application that scales perfectly weakly, the work done by each node remains the same as the scale of the machine increases, which means that we are solving progressively larger problems in the same time as it takes to solve smaller ones on a smaller machine.

Gustafson's law and weak scaling

Amdahl's law gives the upper limit of speedup for a problem of fixed size. This seems to be a bottleneck for parallel computing; if one would like to gain a 500 times speedup on 1000 processors, Amdahl's law requires that the proportion of serial part cannot exceed 0.1%. However, as Gustafson pointed out, in practice the sizes of problems scale with the amount of available resources. If a problem only requires a small amount of resources, it is not beneficial to use a large amount of resources to carry out the computation. A more reasonable choice is to use small amounts of resources for small problems and larger quantities of resources for big problems. Gustafson's law was proposed in 1988, and is based on the approximations that the parallel part scales linearly with the amount of resources, and that the serial part does not increase with respect to the size of the problem. It provides the formula for scaled speedup

$$Speedup = s + p * N$$

where **s**, **p** and **N** have the same meaning as in Amdahl's law. With Gustafson's law the scaled speedup increases linearly with respect to the number of processors (with a slope smaller than one), and there is no upper limit for the scaled speedup. This is called weak scaling, where the scaled speedup is calculated based on the amount of work done for a scaled problem size (in contrast to Amdahl's law which focuses on fixed problem size). In this case the problem size (workload) assigned to each processing element stays constant and additional elements are used to solve a larger total problem (one that wouldn't fit in RAM on a single node, for example). Therefore, this type of measurement is justification for programs that take a lot of memory or other system resources (something that is memory-bound). In the case of weak scaling, linear scaling is achieved if the run time stays constant while the workload is increased in direct proportion to the number of processors. Most programs running in this mode should scale well to larger core counts as they typically employ nearest-neighbour communication patterns where the communication overhead is relatively constant regardless of the number of processes used; exceptions include algorithms that employ heavy use of global communication patterns, eg. FFTs and transposes.

Calculating Weak Scaling Efficiency

If the amount of time to complete a work unit with 1 processing element is **t(1)**, and the amount of time to complete N of the same work units with N processing elements is **t(N)**, the weak scaling efficiency is given as:

$$Efficiency = t(1)/t(N)$$

The concepts of weak and strong scaling are ideals that tend not to be achieved in practice, with real world applications having some of each present. Furthermore, it is the combination of application and computer architecture that determine the type of scaling that occurs. For example, shared memory systems and distributed memory, message passing systems scale differently. Further more, a data parallel application (one in which each node can work on its own separate data set) will by its very nature scale weakly. Before we go on and set you working on some examples of scaling, we should introduce a note of caution. Realistic applications tend to have various levels of complexity and so it may not be obvious just how to measure the increase in “size” of a problem. As an instance, it is known that the solution of a set of N linear equations via Gaussian elimination requires O(N3) floating-point operations (flops). This means that doubling the number of equations does not make the “problem” twice as large, but rather eight times as large! Likewise, if we are solving partial differential equations on a three-dimensional spatial grid and a 1-D time grid, then the problem size would scale like N4.

Measuring parallel scaling performance

When using HPC clusters, it is almost always worthwhile to measure the parallel scaling of your jobs. The measurement of strong scaling is done by testing how the overall computational time of the job scales with the number of processing elements (being either threads or MPI processes), while the test for weak scaling is done by increasing both the job size and the number of processing elements. The results from the parallel scaling tests will provide a good indication of the amount of resources to request for the size of the particular job.

Scaling Measurement Guidelines

Further to basic code performance and optimization concerns (ie. the single thread performance), one should consider the following when timing their application:

```
1. Use wallclock time units or equivalent.
   o eg. timesteps completed per second, etc.
2. Measure using job sizes that span:
   o from 1 to the number of processing elements per node for threaded jobs.
  o from 1 to the total number of processes requested for MPI.
   o job size increments should be in power-of-2 or equivalent (cube powers for weak-scaling 3D simulations,
for example).
   o NOTE: it is inappropriate to refer to scaling numbers with more than 1 cpu as the baseline.
     - in scenarios where the memory requirements exceed what is available on a single node, one should provide
scaling performance for smaller data-sets (lower resolution) so that scaling performance can can be compared
throughout the entire range from 1 to the number of processes they wish to use, or as close to this as
possible, in addition to any results at the desired problem size.
3. Measure multiple independent runs per job size.
  o average results and remove outliers as appropriate.
4. Use a problem state or configuration that best matches your intended production runs.
  o scaling should be measured based on the overall performance of the application.
  o no simplified models or preferential configurations.
5. Various factors must be taken into account when more than one node is used:
  a) Interconnectspeed and latency
  b) Max memory per node
  c) processors per node
```

```
   d) max processors (nodes)
   e) system variables and restrictions (e.g. stacksize)
   NOTE: For applications using MPI the optimization of the MPI settings can also dramatically improve the
application performance. MPI applications also require a certain amount of memory for each MPI   process, which
obbiouvlsy increases with the number of processors and MPI processes used.
6. Additionaly but not necessarily if possible measure using different systems. Most importantly ones that have
significantly different processor / network balances (ie. CPU speed vs. interconnect speed).
   NOTE: The point no 5 as mentioned is not necessary but can be used if code optimization is to be done.
```

Once you have timed your application you should convert the results to scaling efficiencies as explained below. To demonstrate an example for both the weak and strong scaling, a simple example of a conjugate gradient code (https://github.com/yuhlearn/conjugate_gradient) from Github is used. The code is Parallellised using MPI and the user can define N (is the problem size) as the first argument, while executing the code in the terminal. Using N a NxN matrix is generated and filled with values using a random number generator function (rand()) in C. Since this example is used to demonstrate the scaling the output from the code run is not analyzed for correctness but care is taken that the code runs successfully. For strong scaling a problem size N=40000 is choosen and is kept constant while increasing the no of processors. The code is run on the compute nodes of the Noctua at PC2. The details of the nodes are

Noctua, PC2, Paderborn	
CPUs per node	2 (20 cores per CPU)
CPU type	Intel Xeon Gold 6148
main memory per node	192 GiB
interconnect	Intel Omni-Path 100 Gb/s
accelerators used (such as GPUs)	no
number of MPI processes per node	40
number of threads per MPI process (e.g. OpenMP threads)	1

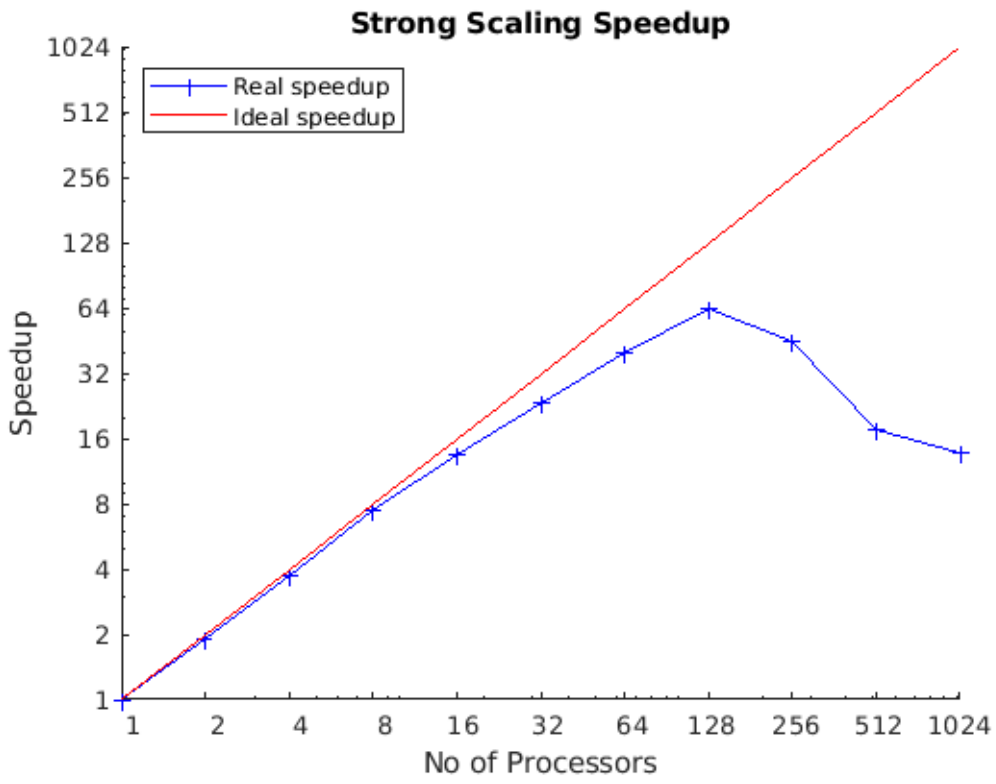
The table below gives a overlook at the speedup for strong scaling acheived for the Conjugate Gradient code.

Strong Scaling

To recall, In case of strong scaling, the number of processors is increased while the problem size remains constant. This also results in a reduced workload per processor.

#problem size (N x N) = 1600000000, where N is Matrix size and N = 40000 for all different processor numbers			
#Processors Np	**#time in seconds T**	**(Amdahl's law) #speeedup = (T(1)/T(Np))**	**#efficiency = (T(1)xN1 / T(Np)xNp)**
1 (1 node)	64.424242	1	1
2 (1 node)	33.901724	1.90	0.95
4 (1 node)	17.449995	3.69	0.92
8 (1 node)	8.734972	7.38	0.92
16 (1 node)	4.789075	13.45	0.84
32 (1 node)	2.749116	23.43	0.73
64 (2 nodes)	1.627157	39.59	0.62
128 (4 nodes)	1.017307	63.33	0.49
256 (7 nodes)	1.436728	44.84	0.18
512 (13 nodes)	3.689217	17.46	0.03
1024 (26 nodes)	4.709213	13.68	0.01
2048 (52 nodes)	21.462228	3.00	0.001

Note: Due to bad Speedup we haven't plotted the row with 2048 processors but it is given for your reference to indicate the trend of decreasing speedup.



The ideal speedup indicates that basically doubling the processor no the speedup should be doubled but with the real speedup we see the speedup we get in the table above. For user information the efficieny for the strong scaling is also calculated with the formula mentioned at the top of the table but is not plotted. For project applications specifying the speedup is sufficient. As we mentioned earlier we want to locate a sweet spot for the code run and we see from the plot and the table that fort he given problem size using 4 nodes (128 cores) a maximum speedup is acheived after which a decline in speedup is observed. The decline in speedup and deviation in real speedup from ideal speedup for different no of processor runs can be attributed due to following reasons –

1) The difference between Amdahl's Law (Ideal speedup) and real-speedup is that Amdahl's Law assumes an infinitely fast network. That is, data can be transferred infinitely fast from one process to another (zero latency and infinite bandwidth). In real life, this isn't true. All networks have non-zero latency and limited bandwidth, so data is not communicated infinitely fast but in some finite amount of time. Another way to view Amdahl's law is as a guide to where you should concentrate your resources to improve performance. I'm going to assume you have done everything you can to improve the parallel portion of the code. You are using asynchronous data transfers, you limit the number of collective operations, you are using a very low latency and high bandwidth network, you have profiled your application to understand the MPI portion, and so on (please ignore if you don't understand all terms). At this point, Amdahl's Law says that to get better performance, you need to focus on the serial portion of your application. Remember how far you can scale your parallel application is driven by the serial portion of your application.

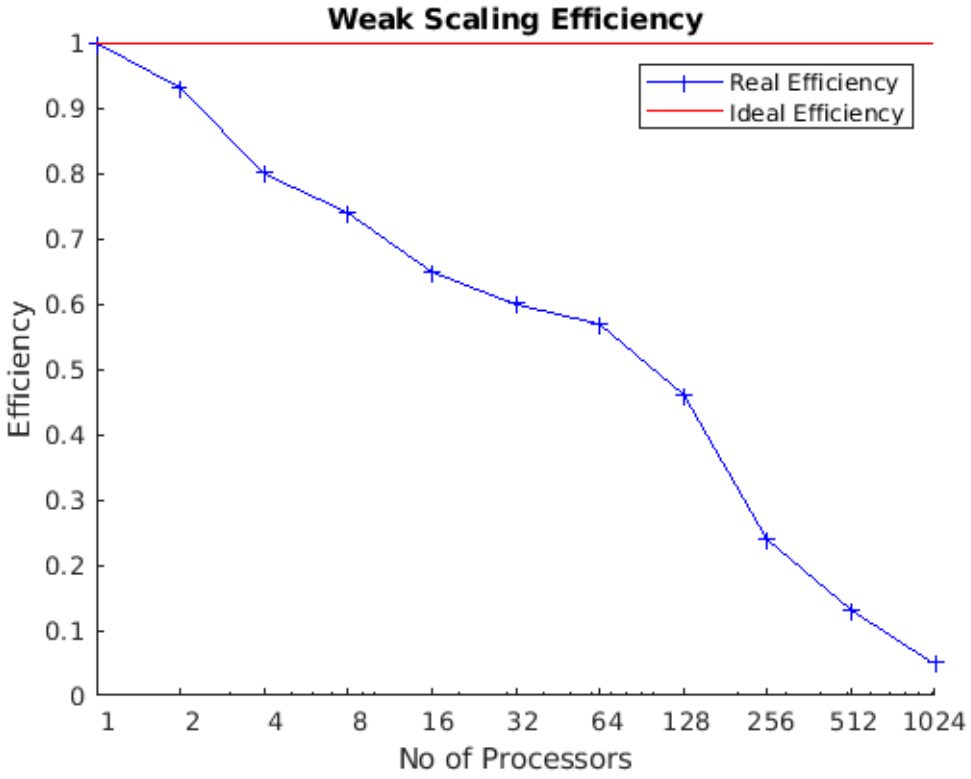
2) For real speedup the speedup after 4 nodes gets worse. Because of increasing data traffic between the compute nodes, adding another node can be a disadvantage and will even slow down the application. It can also be so said that the compute time also cannot be reduced further as the problem size cannot be further divided to improve it for a effective reduction in time for data traffic and I/O.

Weak Scaling

To recall, In case of weak scaling, both the number of processors and the problem size are increased. This also results in a constant workload per processor.

#problem size (N x N), where N is Matrix size and N is different for all different processor numbers an given in table below			
#Processors Np	**#time in seconds T**	**#weak scaling efficiency = (T(1)/T(Np))**	**#problem size (N X N), N is Matrix size**
1 (1 node)	0.582497	1	16000000
2 (1 node)	0.627571	0.93	32000000
4 (1 node)	0.72413	0.80	64000000
8 (1 node)	0.787381	0.74	128000000
16 (1 node)	0.893998	0.65	256000000
32 (1 node)	0.968348	0.60	512000000
64 (2 nodes)	1.030421	0.57	1024000000
128 (4 nodes)	1.257982	0.46	2048000000
256 (7 nodes)	2.408056	0.24	4096000000
512 (13 nodes)	4.593224	0.13	8192000000
1024 (26 nodes)	10.693525	0.05	16384000000
2048 (52 nodes)	22.014061	0.03	32768000000

Note: Due to low Efficiency we haven't plotted the row with 2048 processors but it is given for your reference to indicate the trend of decreasing efficiency.



The Efficiency formulation given at top of weak scaling table is used to calculate the efficiencies with increasing no of processors. The problem size is increased in proportion to the no of processors starting with a problem size of 16000000 (N X N, where N=4000 for 1 processor). This problem size is doubled, while the no of processors is doubled, such that the compute load for each processor is same with increasing processors. The Ideal efficiency would than stay one as it says the same compute work needs to be done for different no of processors, resulting in same wallclock time as for 1 processor. But the real efficiency shows a decrease in efficiency with increasing no of processors. The trend in the graph for real efficiency can be explained as below –

1) For runs within one node as problem size increases with increase in no of processors from 1 processor time is spent in MPI setup, more data needs to be communicated between the processors this increases the latency in communication for the limited bandwidth of communication network.

2) For more than one nodes, again due to increasing problem size the latency of data transfer between the nodes increases for the limited bandwidth of communication network. Also the another factor that may play a part is load balancing between the nodes, since this is a basic implementation of CG algorithm without much code optimization for parallel runs.

3) Granularity which defines the frequency of communication between parallel jobs (strong granularity means more frequency of communication) is unknown for the CG code but if we assume that it has strong granularity than certainly it means more latency besides the I/O bottlenecks due to more data. For massive problems (such as global satellite imagery analytics or streaming data problems) weak scaling can be a good evaluation criteria because it keeps the problem size per processor constant, helping to evaluate how well a parallel application can scale to a massive spatial problem.

Rules for presenting performance results

In the intro to the book Performance Tuning of Scientific Applications (https://www.researchgate.net/publication/262602472_Performance_Tuning_of_Scientific_Applications), David Bailey suggests nine guidelines for presenting performance results without misleading the reader. Paraphrasing only slightly, these are:

```
1. Follow rules on benchmarks.
2. Only present actual performance, not extrapolations.
3. Compare based on comparable levels of tuning.
4. Compare wall clock times (not flop rates).
5. Compute performance rates from consistent operation counts based on the best serial codes.
6. Speedup should compare to best serial version. Scaled speedup plots should be clearly labeled and explained.
7. Fully disclose information affecting performance: 32/64 bit, use of assembly, timing of a subsystem rather
than the full system, etc.
8. Don’t deceive skimmers. Take care not to make graphics, figures, and abstracts misleading, even in
isolation.
9. Report enough information to allow others to reproduce the results. If possible, this should include
   o The hardware, software and system environment
   o The language, algorithms, data types, and coding techniques used
   o The nature and extent of tuning
   o The basis for timings, flop counts, and speedup computations
```

Summary

This post discusses two common types of scaling of software: strong scaling and weak scaling. Some key points are summarized below.

• Scalability is important for parallel computing to be efficient.

• Strong scaling concerns the speedup for a fixed problem size with respect to the number of processors, and is governed by Amdahl's law.

• Weak scaling concerns the speedup for a scaled problem size with respect to the number of processors, and is governed by Gustafson's law.

• When using HPC clusters, it is almost always worthwhile to measure the scaling of your jobs.

• The results of strong and weak scaling tests provide good indications for the best match between job size and the amount of resources (for resource planning) that should be requested for a particular job.

• Scalability testing measures the ability of an application to perform well or better with varying problem sizes and numbers of processors. It does not test the applications general funcionality or correctness.

References

The PDC Center for High Performance Computing Blog at the KTH Royal Institute of Technology (https://www.kth.se/blogs/pdc/)

Introduction to High Performance Computing for Scientists and Engineers (https://blogs.fau.de/hager/hpc-book)

Performance Tuning of Scientific Applications (https://www.researchgate.net/publication/262602472_Performance_Tuning_of_Scientific_Applications)

Measuring Parallel Scaling Performance (https://www.sharcnet.ca/help/index.php/Measuring_Parallel_Scaling_Performance)

CSE Lab ETH Zuerich Presentation on Strong and Weak Scaling (https://www.cse-lab.ethz.ch/wp-content/uploads/2018/11/amdahl_gustafson.pdf)